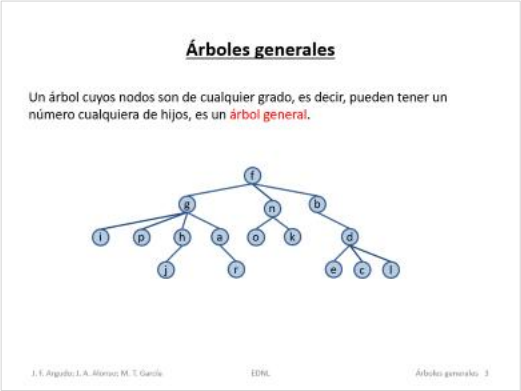
Estructuras de Datos no Lineales

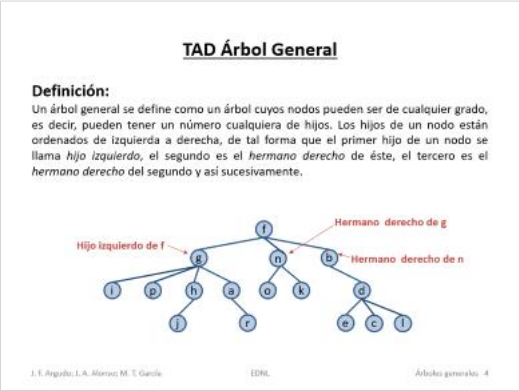
1.3. Árboles generales

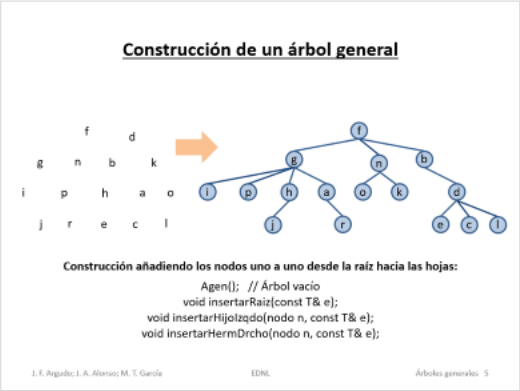
José Fidel Argudo Argudo
José Antonio Alonso de la Huerta
Mª Teresa García Horcajadas

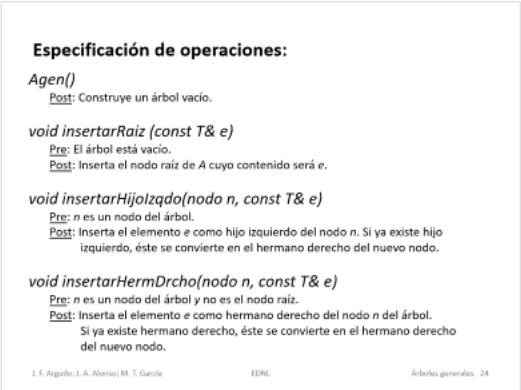
Versión 2.4

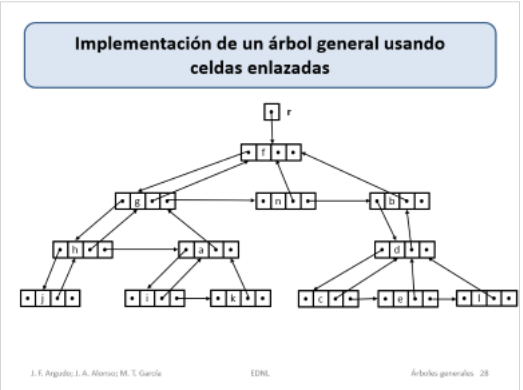
Contenido

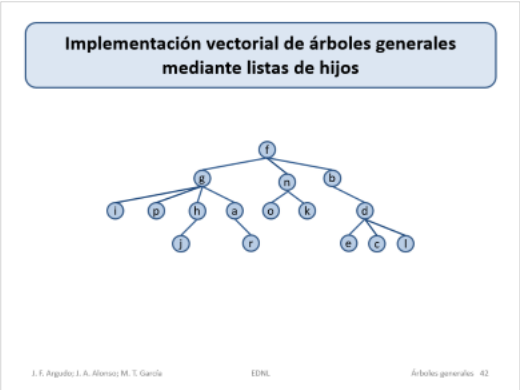

Árboles generales
Un árbol cuyos nodos son de cualquier grado, es decir, pueden tener un número cualquiera de hijos, es un árbol general.
J. F. Argudo; J. A. Alonso; M. T. García
EDNL
Árboles generales 3


TAD Árbol General
Definición:
Un árbol general se define como un árbol cuyos nodos pueden ser de cualquier grado, es decir, pueden tener un número cualquiera de hijos. Los hijos de un nodo están ordenados de izquierda a derecha, de tal forma que el primer hijo de un nodo se llama hijo izquierdo, el segundo es el hermano derecho de éste, el tercero es el hermano derecho del segundo y así sucesivamente.
Hijo izquierdo de f
Hermano derecho de g
Hermano derecho de n
J. F. Argudo; J. A. Alonso; M. T. García
EDNL
Árboles generales 4


Construcción de un árbol general
Construcción añadiendo los nodos uno a uno desde la raíz hacia las hojas:
Agen(); // Árbol vacío
void insertarRaiz(const T& e);
void insertarHijoIzqdo(nodo n, const T& e);
void insertarHermDrcho(nodo n, const T& e);
J. F. Argudo; J. A. Alonso; M. T. García
EDNL
Árboles generales 5

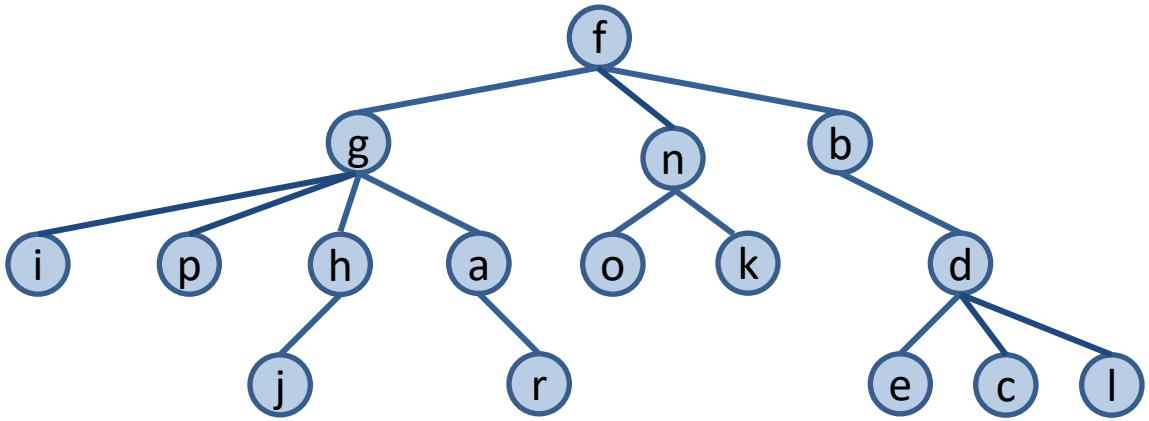

Especificación de operaciones:
Agen()
Post: Construye un árbol vacío.
void insertarRaiz (const T& e)
Pre: El árbol está vacío.
Post: Inserta el nodo raíz de A cuyo contenido será e.
void insertarHijoIzqdo(nodo n, const T& e)
Pre: n es un nodo del árbol.
Post: Inserta el elemento e como hijo izquierdo del nodo n. Si ya existe hijo izquierdo, éste se convierte en el hermano derecho del nuevo nodo.
void insertarHermDrcho(nodo n, const T& e)
Pre: n es un nodo del árbol y no es el nodo raíz.
Post: Inserta el elemento e como hermano derecho del nodo n del árbol. Si ya existe hermano derecho, éste se convierte en el hermano derecho del nuevo nodo.
J. F. Argudo; J. A. Alonso; M. T. García
EDNL
Árboles generales 24


Implementación de un árbol general usando celdas enlazadas
J. F. Argudo; J. A. Alonso; M. T. García
EDNL
Árboles generales 28


Implementación vectorial de árboles generales mediante listas de hijos
J. F. Argudo; J. A. Alonso; M. T. García
EDNL
Árboles generales 42

Árboles generales

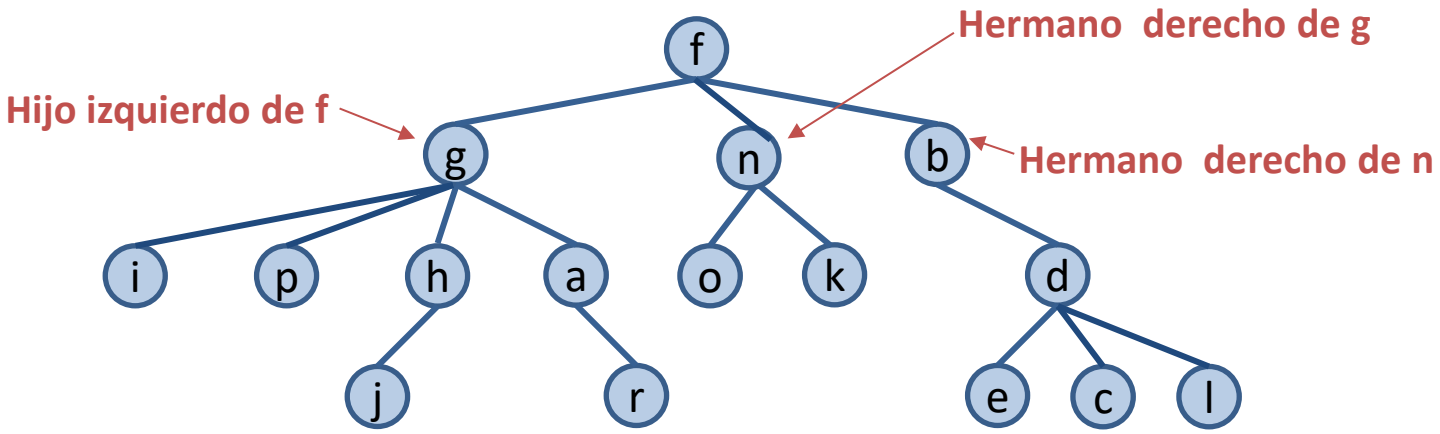
Un árbol cuyos nodos son de cualquier grado, es decir, pueden tener un número cualquiera de hijos, es un árbol general.



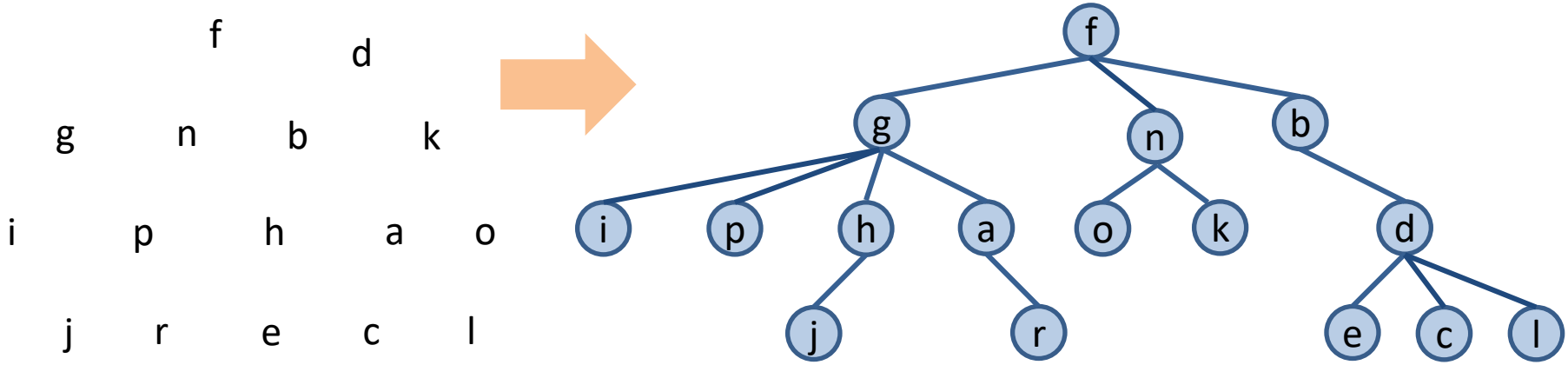
TAD Árbol General

Definición:

Un árbol general se define como un árbol cuyos nodos pueden ser de cualquier grado, es decir, pueden tener un número cualquiera de hijos. Los hijos de un nodo están ordenados de izquierda a derecha, de tal forma que el primer hijo de un nodo se llama *hijo izquierdo*, el segundo es el *hermano derecho* de éste, el tercero es el *hermano derecho* del segundo y así sucesivamente.



Construcción de un árbol general



Construcción añadiendo los nodos uno a uno desde la raíz hacia las hojas:

Agen(); // Árbol vacío
void insertarRaiz(const T& e);
void insertarHijoIzqdo(nodo n, const T& e);
void insertarHermDrcho(nodo n, const T& e);

Construcción de un árbol general

Creación del árbol A como un contenedor vacío.

A

f d

g n b k

i p h a o

j r e c l

Agen A;

Construcción de un árbol general

d

g n b k

i p h a o

j r e c l

A

A.insertarRaiz(‘f’);

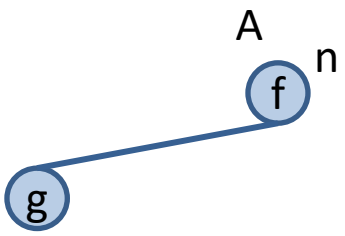
Construcción de un árbol general

d

n b k

i p h a o

j r e c l



A.insertarHijoIzqdo(n, ‘g’);

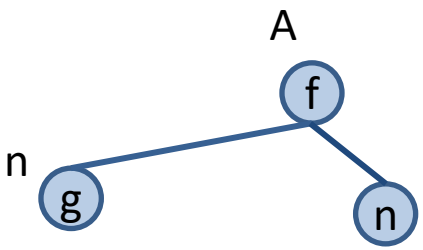
Construcción de un árbol general

d

b k

i p h a o

j r e c l



A.insertarHermDrcho(n, ‘n’);

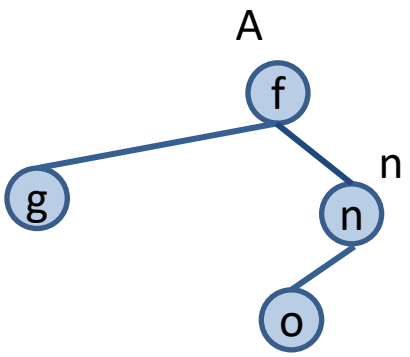
Construcción de un árbol general

d

b k

i p h a

j r e c l



A.insertarHijoIzqdo(n, ‘o’);

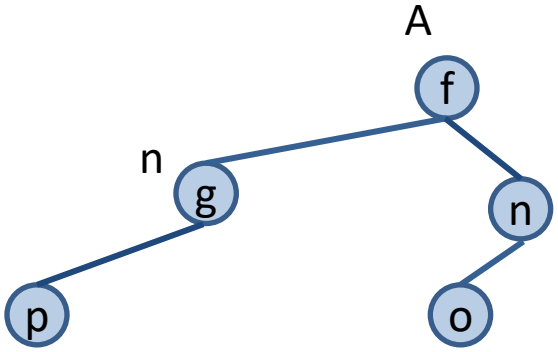
Construcción de un árbol general

d

b k

i h a

j r e c l



A.insertarHijoIzqdo(n, ‘p’);

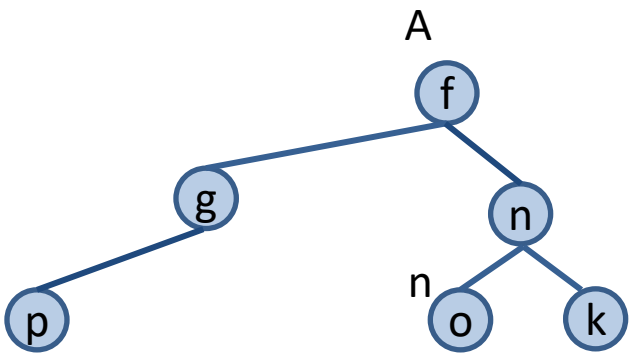
Construcción de un árbol general

d

b

i h a

j r e c l



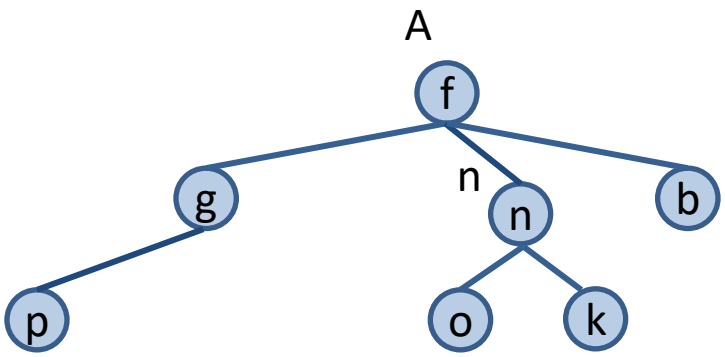
A.insertarHermDrcho(n, ‘k’);

Construcción de un árbol general

d

i h a

j r e c l

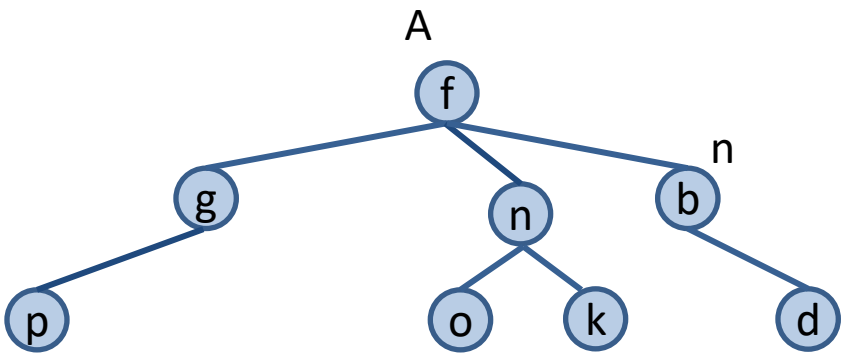


A.insertarHermDrcho(n, ‘b’);

Construcción de un árbol general

i h a

j r e c l

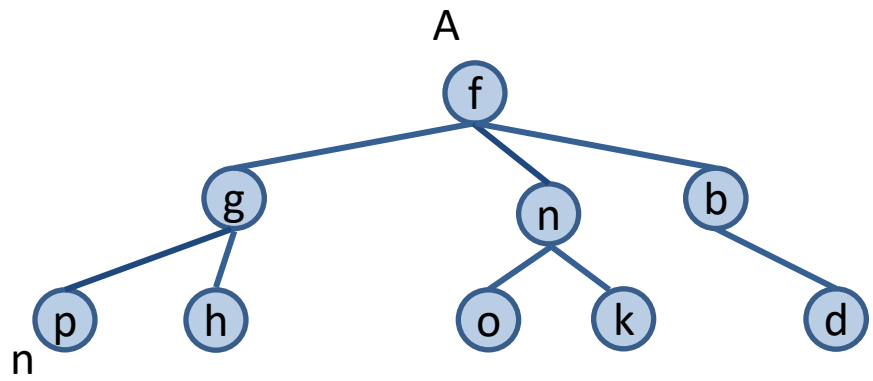


A.insertarHijoIzqdo(n, ‘d’);

Construcción de un árbol general

i a

j r e c l

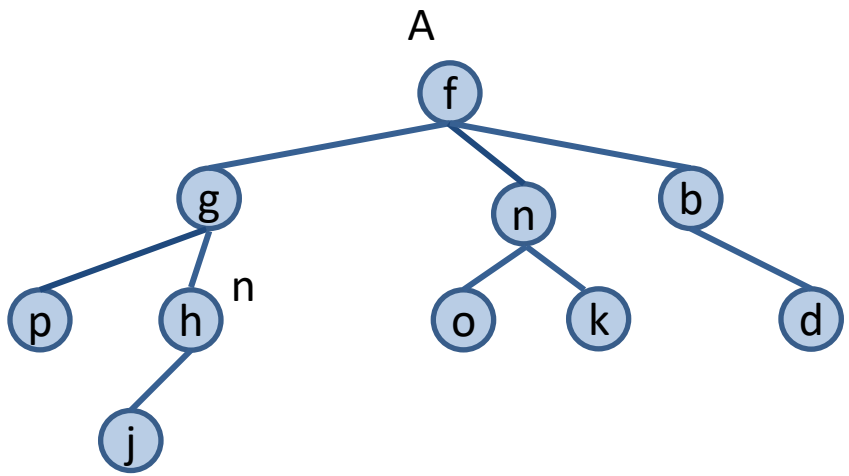


A.insertarHermDrcho(n, ‘h’);

Construcción de un árbol general

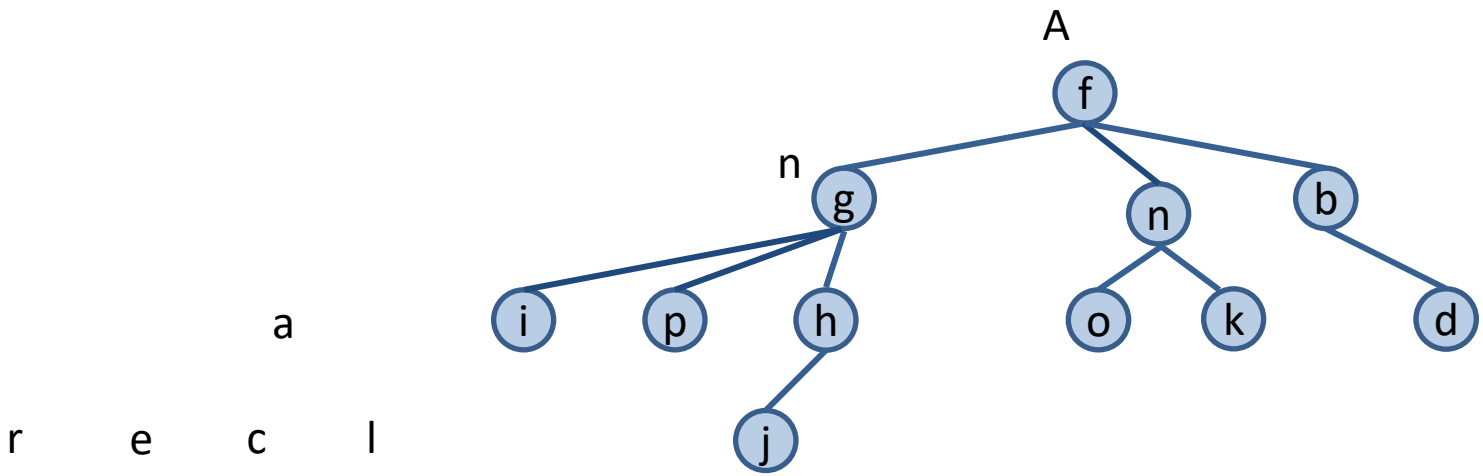
i a

r e c l



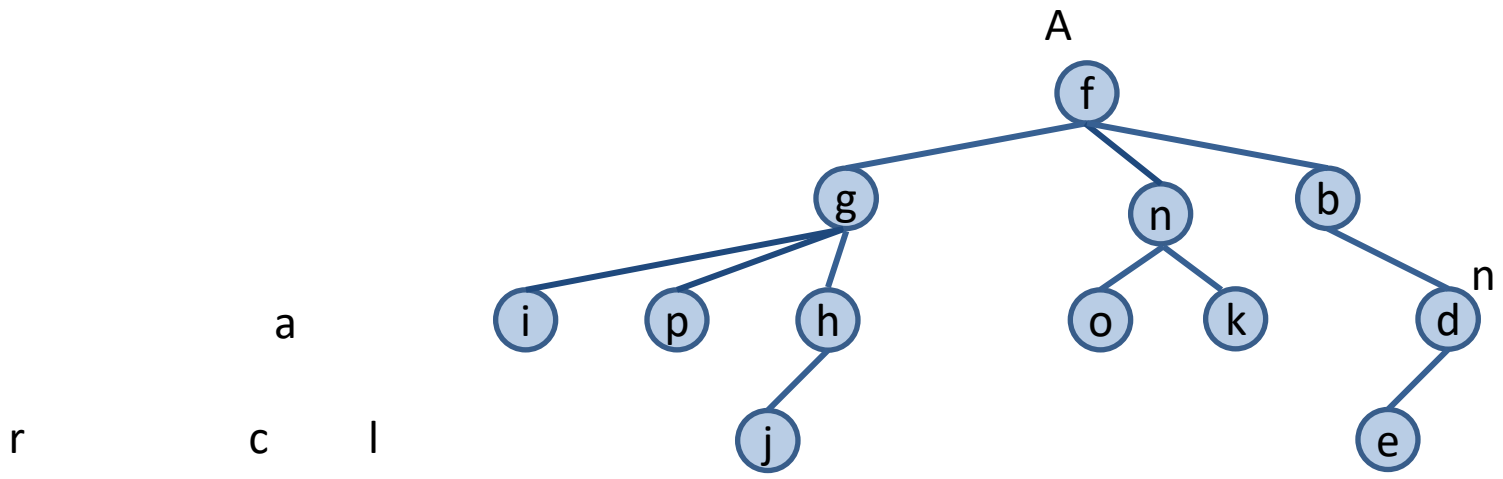
A.insertarHijoIzqdo(n, ‘j’);

Construcción de un árbol general



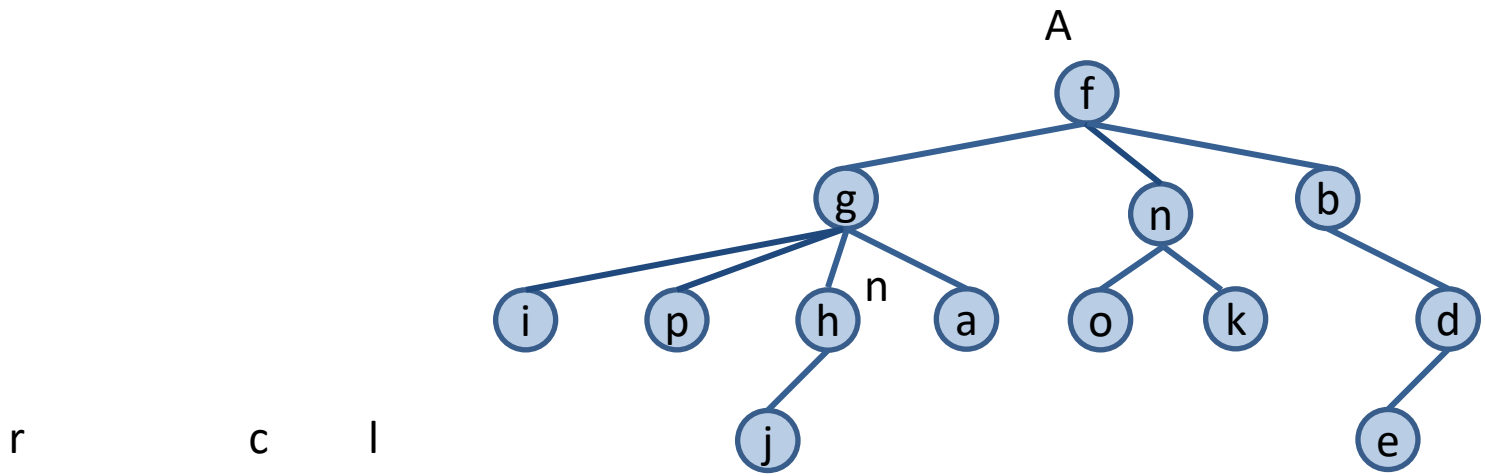
A.insertarHijoIzqdo(n, ‘i’);

Construcción de un árbol general



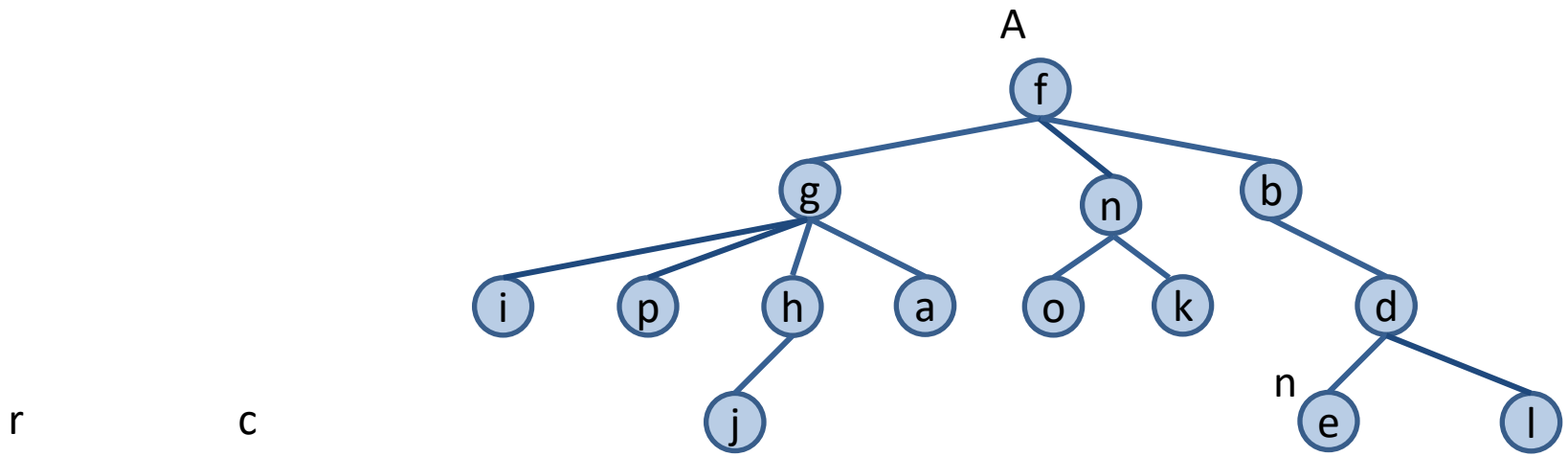
A.insertarHijoIzqdo(n, ‘e’);

Construcción de un árbol general



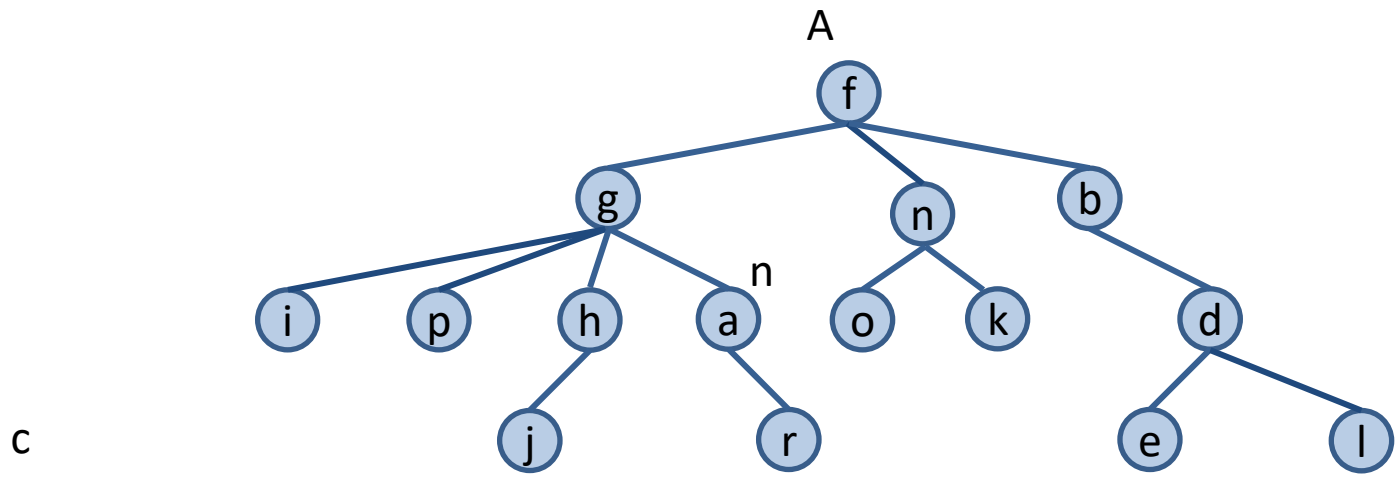
A.insertarHermDrcho(n, ‘a’);

Construcción de un árbol general



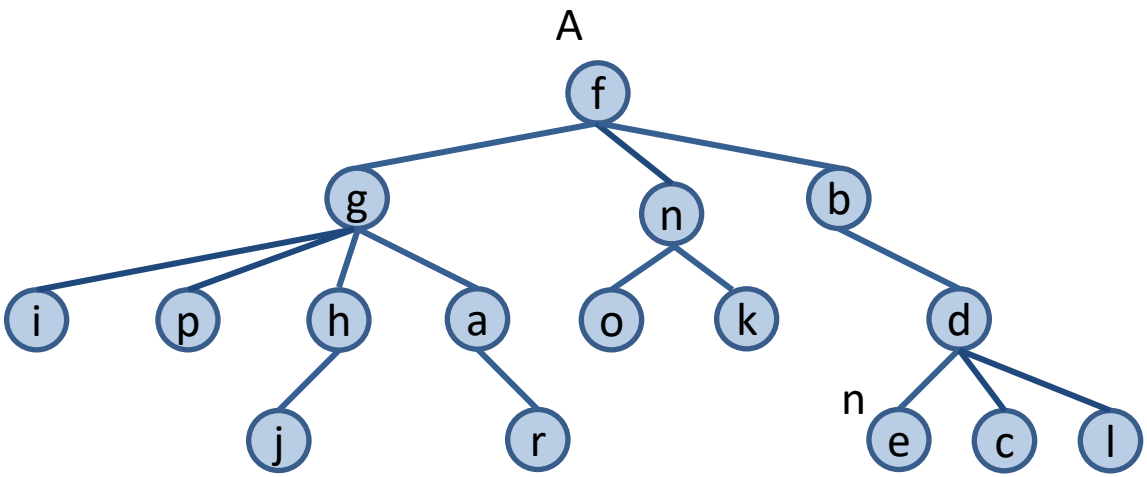
A.insertarHermDrcho(n, ‘l’);

Construcción de un árbol general



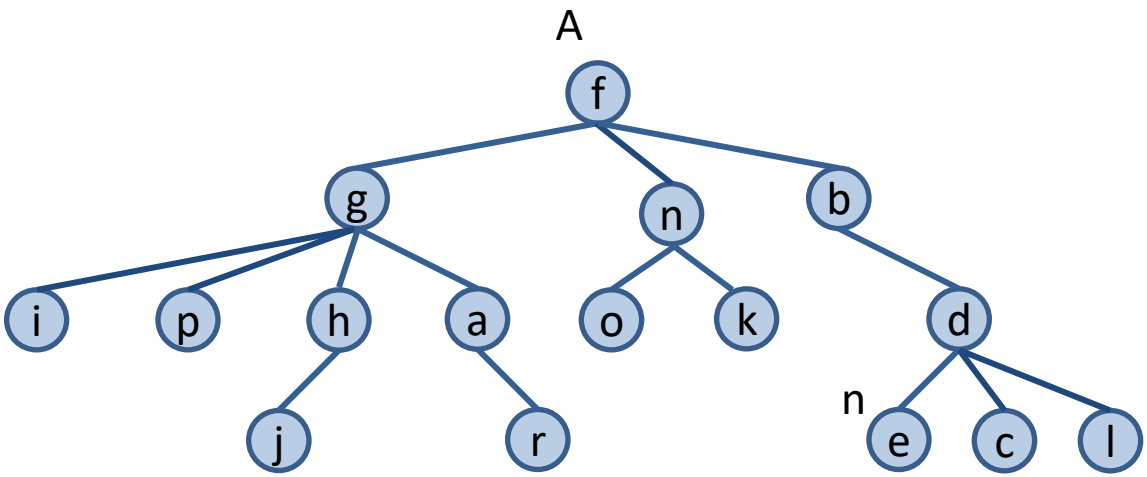
A.insertarHijoIzqdo(n, ‘r’);

Construcción de un árbol general



A.insertarHermDrcho(n, ‘c’);

Construcción de un árbol general



Especificación de operaciones:

Agen()

Post: Construye un árbol vacío.

void insertarRaiz (const T& e)

Pre: El árbol está vacío.

Post: Inserta el nodo raíz de *A* cuyo contenido será *e*.

void insertarHijoIzqdo(nodo n, const T& e)

Pre: *n* es un nodo del árbol.

Post: Inserta el elemento *e* como hijo izquierdo del nodo *n*. Si ya existe hijo izquierdo, éste se convierte en el hermano derecho del nuevo nodo.

void insertarHermDrcho(nodo n, const T& e)

Pre: *n* es un nodo del árbol *y* no es el nodo raíz.

Post: Inserta el elemento *e* como hermano derecho del nodo *n* del árbol. Si ya existe hermano derecho, éste se convierte en el hermano derecho del nuevo nodo.

void eliminarHijoIzqdo(nodo n)

Pre: *n* es un nodo del árbol. Existe *hijoIzqdo(n)* y es una hoja.

Post: Destruye el hijo izquierdo del nodo *n*. El segundo hijo, si existe, se convierte en el nuevo hijo izquierdo de *n*.

void eliminarHermDrcho(nodo n)

Pre: *n* es un nodo del árbol. Existe *hermDrcho(n)* y es una hoja.

Post: Destruye el hermano derecho del nodo *n*. El siguiente hermano se convierte en el nuevo hermano derecho de *n*.

void eliminarRaiz()

Pre: El árbol no está vacío y *raiz()* es una hoja.

Post: Destruye el nodo raíz. El árbol queda vacío.

bool vacio () const

Post: Devuelve **`true`** si el árbol está vacío y **`false`** en caso contrario.

size_t tama () const

Post: Devuelve el número de elementos que contiene el árbol.

const T& elemento(nodo n) const

T& elemento(nodo n)

Pre: *n* es un nodo del árbol.

Post: Devuelve el elemento del nodo *n*.

nodo raiz() const

Post: Devuelve el nodo raíz del árbol. Si el árbol está vacío, devuelve *NODO_NULO*.

nodo padre(nodo n) const

Pre: *n* es un nodo del árbol.

Post: Devuelve el padre del nodo *n*. Si *n* es el nodo raíz, devuelve *NODO_NULO*.

nodo hijoIzqdo(nodo n) const

Pre: *n* es un nodo del árbol.

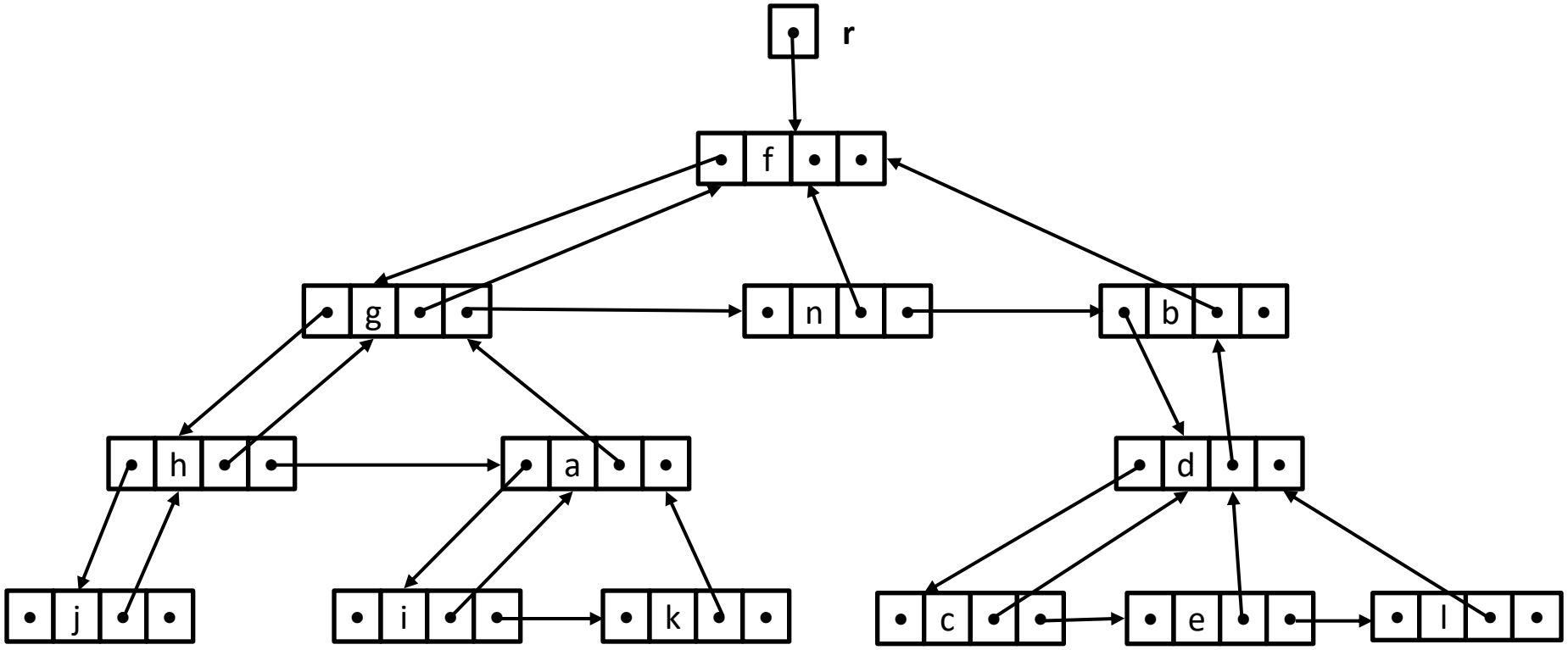
Post: Devuelve el hijo izquierdo del nodo *n*. Si no existe, devuelve *NODO_NULO*.

nodo hermDrcho(nodo n) const

Pre: *n* es un nodo del árbol.

Post: Devuelve el hermano derecho del nodo *n*. Si no existe, devuelve N*ODO_NULO*.

Implementación de un árbol general usando celdas enlazadas



```
#ifndef AGEN_H
#define AGEN_H
#include <cassert>
#include <cstddef>   // size_t
#include <utility>   // swap

template <typename T> class Agen {
   struct celda;   // Declaración adelantada privada
public:
   typedef celda* nodo;
   static const nodo NODO_NULO;

   Agen();
   void insertarRaiz(const T& e);
   void insertarHijoIzqdo(nodo n, const T& e);
   void insertarHermDrcho(nodo n, const T& e);
   void eliminarHijoIzqdo(nodo n);
   void eliminarHermDrcho(nodo n);
   void eliminarRaiz();
   bool vacio() const;
   size_t tama() const;
   const T& elemento(nodo n) const;        // Lectura en const Agen
   T& elemento(nodo n);     // Lectura/escritura en Agen (no-const)
```

```
   nodo raiz() const;
   nodo padre(nodo n) const;
   nodo hijoIzqdo(nodo n) const;
   nodo hermDrcho(nodo n) const;
   Agen(const Agen& A);                    // Ctor. de copia
   Agen& operator =(const Agen& A);        // Asignación de árboles
   ~Agen();                                // Destructor
private:
   struct celda {
      T elto;
      nodo padre, hizq, heder;
   };

   nodo r;           // Raíz del árbol
   size_t numNodos; // Tamaño del árbol

   nodo copiar(nodo n);
   void destruir(nodo& n);
};   // class Agen
```

```
// Definición del nodo nulo
template <typename T>
const typename Agen<T>::nodo Agen<T>::NODO_NULO{nullptr};

/*-----------------------------------------------------------*/
/* Métodos públicos                                          */
/*-----------------------------------------------------------*/

template <typename T>
inline Agen<T>::Agen() : r{NODO_NULO}, numNodos{0} {}

template <typename T>
inline void Agen<T>::insertarRaiz(const T& e)
{
   assert(vacio());

   r = new celda{e}; // padre, hizq, heder == nullptr == NODO_NULO
   numNodos = 1;
}
```

```
template <typename T>
inline void Agen<T>::insertarHijoIzqdo(nodo n, const T& e)
{
   assert(n != NODO_NULO);

   // Hijo izqdo. actual se convierte en hermano drcho. del nuevo hijo
   n->hizq = new celda{e, n, NODO_NULO, n->hizq};
   ++numNodos;
}

template <typename T>
inline void Agen<T>::insertarHermDrcho(nodo n, const T& e)
{
   assert(n != NODO_NULO);
   assert(n != r);   // n no es la raíz.

   // Hermano drcho. actual se convierte en hermano drcho. del nuevo.
   n->heder = new celda{e, n->padre, NODO_NULO, n->heder};
   ++numNodos;
}
```

```
template <typename T>
inline void Agen<T>::eliminarHijoIzqdo(nodo n)
{
   nodo hizqdo;

   assert(n != NODO_NULO);
   hizqdo = n->hizq;
   assert(hizqdo != NODO_NULO);        // Existe hijo izqdo.
   assert(hizqdo->hizq == NODO_NULO); // y es hoja.

   // El hermano drcho. pasa a ser el nuevo hijo izqdo.
   n->hizq = hizqdo->heder;
   delete hizqdo;
   --numNodos;
}
```

```
template <typename T>
inline void Agen<T>::eliminarHermDrcho(nodo n)
{
   nodo hedrcho;

   assert(n != NODO_NULO);
   hedrcho = n->heder;
   assert(hedrcho != NODO_NULO);   // Existe hermano drcho.
   assert(hedrcho->hizq == NODO_NULO); // y es hoja.

   // El hermano del hermano se convierte en el
   // nuevo hermano drcho. de n.
   n->heder = hedrcho->heder;
   delete hedrcho;
   --numNodos;
}
```

```
template <typename T>
inline void Agen<T>::eliminarRaiz()
{
   assert(numNodos == 1);

   delete(r);
   r = NODO_NULO;
   numNodos = 0;
}
```

```
template <typename T>
inline bool Agen<T>::vacio() const
{ return numNodos == 0; }

template <typename T>
inline size_t Agen<T>::tama() const
{ return numNodos; }

template <typename T>
inline const T& Agen<T>::elemento(nodo n) const
{
   assert(n != NODO_NULO);
   return n->elto;
}

template <typename T>
inline T& Agen<T>::elemento(nodo n)
{
   assert(n != NODO_NULO);
   return n->elto;
}
```

```
template <typename T>
inline typename Agen<T>::nodo Agen<T>::raiz() const
{
   return r;
}

template <typename T> inline
typename Agen<T>::nodo Agen<T>::padre(nodo n) const
{
   assert(n != NODO_NULO);

   return n->padre;
}
```

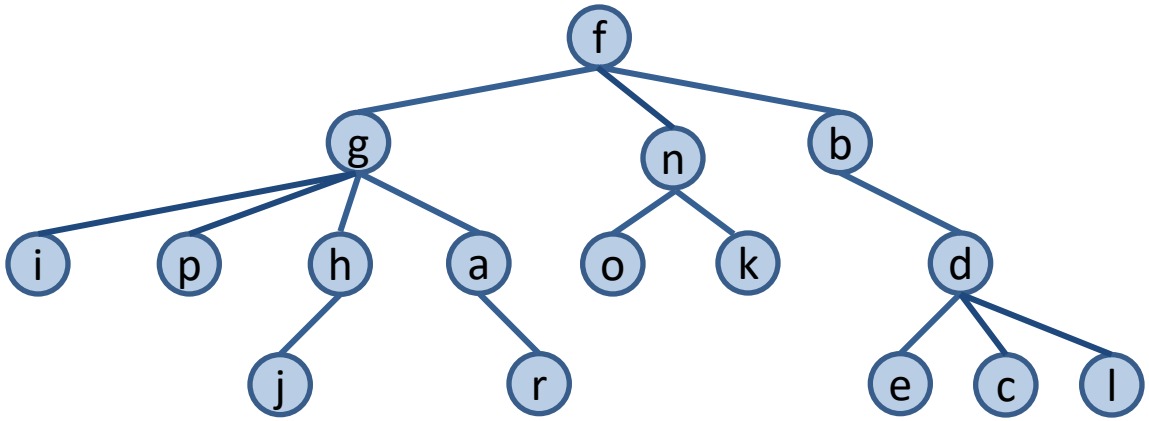
```
template <typename T> inline
typename Agen<T>::nodo Agen<T>::hijoIzqdo(nodo n) const
{
   assert(n != NODO_NULO);

   return n->hizq;
}

template <typename T> inline
typename Agen<T>::nodo Agen<T>::hermDrcho(nodo n) const
{
   assert(n != NODO_NULO);

   return n->heder;
}
```

```
template <typename T>
inline Agen<T>::Agen(const Agen& A) :
   Agen{} // Garantiza destrucción de copia incompleta de A.
{
   r = copiar(A.r);   // Copiar raíz y descendientes.
   numNodos = A.numNodos;
}

template <typename T>
inline Agen<T>& Agen<T>::operator =(const Agen& A)
{
   Agen B{A};
   std::swap(r, B.r);
   std::swap(numNodos, B.numNodos);
   return *this;
}

template <typename T>
inline Agen<T>::~Agen()
{
   destruir(r); // Vaciar el árbol.
}
```

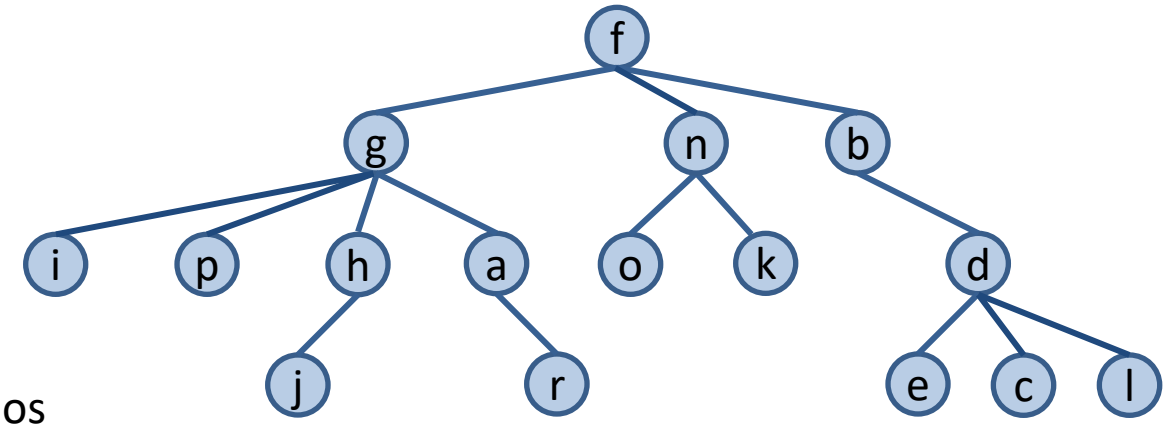
```
/*-----------------------------------------------------------*/
/* Métodos privados                                          */
/*-----------------------------------------------------------*/
// Destruye un nodo y todos sus descendientes
template <typename T>
void Agen<T>::destruir(nodo& n)
{
   nodo hedrcho;

   if (n != NODO_NULO)
   {
      while (n->hizq != NODO_NULO)
      {  // Destruir hijos
         hedrcho = n->hizq->heder;
         destruir(n->hizq);
         n->hizq = hedrcho;
      }
      delete n;
      n = NODO_NULO;
   }
}
```

```
// Devuelve una copia de un nodo y todos sus descendientes
template <typename T>
typename Agen<T>::nodo Agen<T>::copiar(nodo n)
{
   nodo m = NODO_NULO;
   if (n != NODO_NULO) {
      Agen A; // Contiene los nodos copiados.
              // Si la copia no se completa, A es destruido.
      A.r = new celda{n->elto};     // Copiar n en raíz.
      if (n->hizq != NODO_NULO) {   // n tiene descendientes.
         A.r->hizq = copiar(n->hizq); // Copiar primer subárbol.
         A.r->hizq->padre = A.r;
         // Copiar el resto de subárboles.
         nodo hijo = A.r->hizq;          // Último subárbol copiado.
         nodo hedrcho = n->hizq->heder; // Siguiente subárbol a copiar
         while (hedrcho != NODO_NULO) {
            hijo = hijo->heder = copiar(hedrcho);
            hijo->padre = A.r;
            hedrcho = hedrcho->heder;
         }
      }
      m = A.r; A.r = NODO_NULO; // Evita destruir la copia.
   }
   return m;
}
#endif // AGEN_H
```

Implementación vectorial de árboles generales mediante listas de hijos

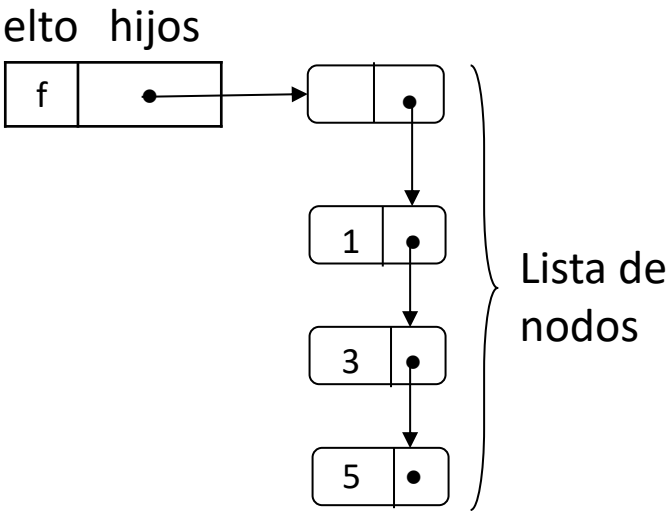


Implementación vectorial de árboles generales mediante listas de hijos

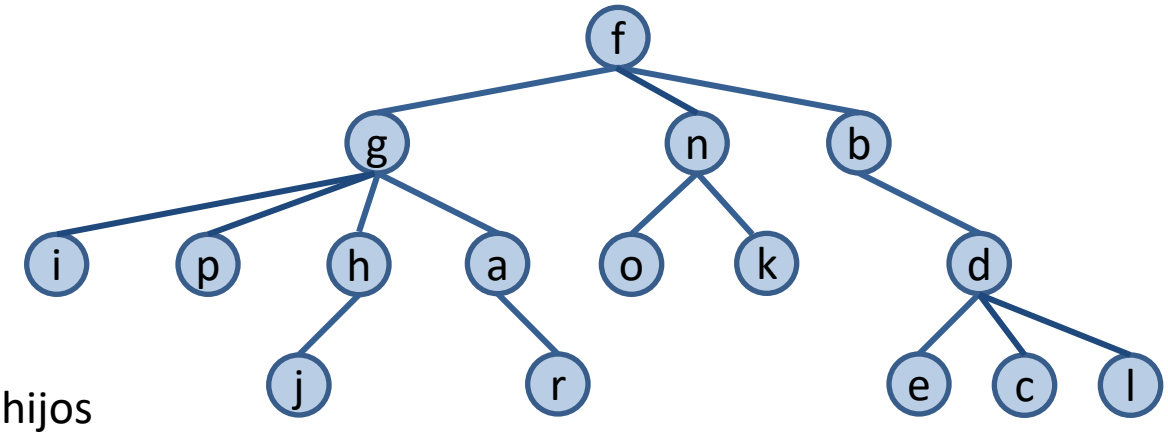


	elto	hijos
0	f	(1,3,5)
1	g	(7,14,2,8)
2	h	(9)
3	n	(4,10)
4	o	( )
5	b	(6)
6	d	(11,12,13)
..		
99	—	—

nodos	•
numNodos	16
maxNodos	100

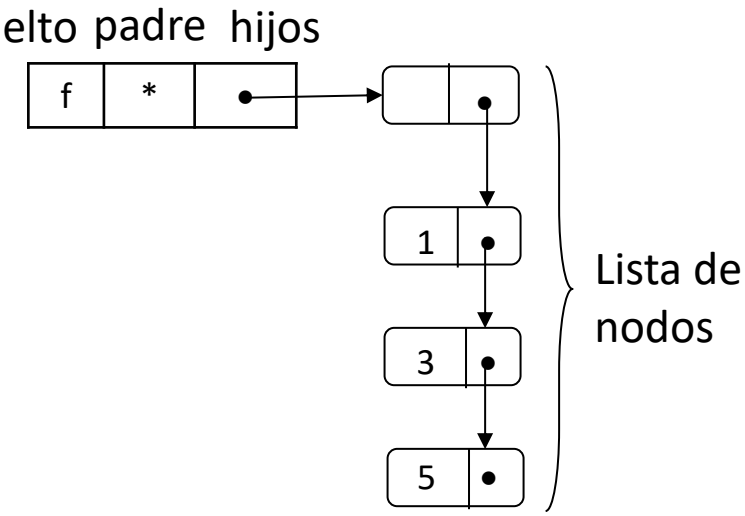


Implementación vectorial de árboles generales mediante listas de hijos



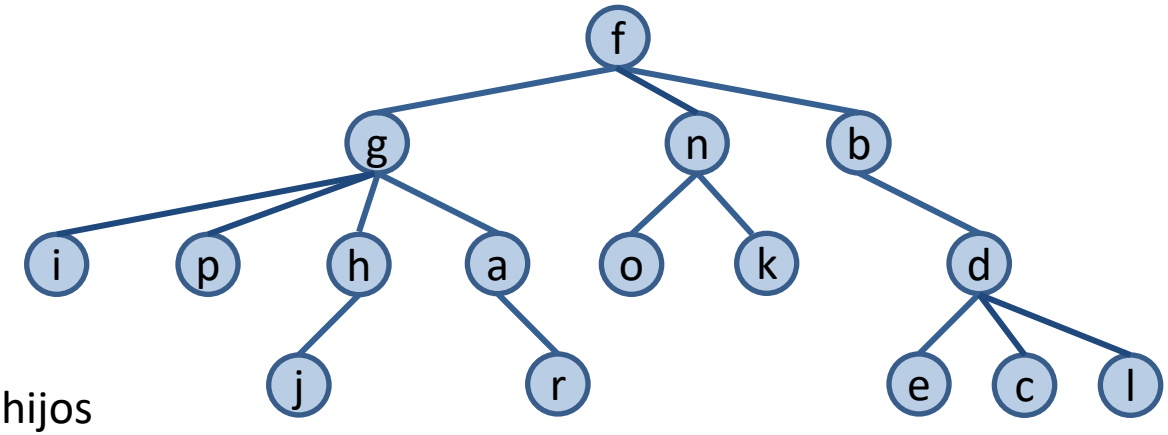
	elto	padre	hijos
0	f	*	(1,3,5)
1	g	0	(7,14,2,8)
2	h	1	(9)
3	n	0	(4,10)
4	o	3	( )
5	b	0	(6)
6	d	5	(11,12,13)
..			
99	—	*	—

nodos	•
numNodos	16
maxNodos	100



Implementación vectorial de árboles generales mediante listas de hijos

(gestión de celdas libres)



	elto	padre	hijos
0	f	*	(1,3,5)
1	g	0	(7,14,2,8)
2	h	1	(9)
…			
25	—	*	(26)
26	—	*	(99)
27	—	*	(maxNodos)
…			
99	—	*	(27)

Fin de lista

nodos	•
numNodos	16
maxNodos	100
libre	25

Lista de celdas libres enlazadas por el hijo izquierdo

```
#ifndef AGEN_LIS_H
#define AGEN_LIS_H
#include <cassert>
#include <cstddef>   // size_t
#include <cstdint>   // SIZE_MAX, máx. de size_t
#include <utility>   // swap
#include "listaenla.h"

template <typename T> class Agen {
public:
   typedef size_t nodo;
   static const nodo NODO_NULO;
   explicit Agen(size_t maxNodos = 0);
   void insertarRaiz(const T& e);
   void insertarHijoIzqdo(nodo n, const T& e);
   void insertarHermDrcho(nodo n, const T& e);
   void eliminarHijoIzqdo(nodo n);
   void eliminarHermDrcho(nodo n);
   void eliminarRaiz();
   bool vacio() const;
```

```
   size_t tama() const;
   size_t tamaMax() const;        // Requerido por la implementación
   const T& elemento(nodo n) const;        // Lectura en const Agen
   T& elemento(nodo n);       // Lectura/escritura en Agen (no-const)
   nodo raiz() const;
   nodo padre(nodo n) const;
   nodo hijoIzqdo(nodo n) const;
   nodo hermDrcho(nodo n) const;
   Agen(const Agen& A);                    // Ctor. de copia
   Agen& operator =(const Agen& A);        // Asignación de árboles
   ~Agen();                                // Destructor
private:
   struct celda {
      T elto;
      nodo padre;
      Lista<nodo> hijos;
   };
   celda* nodos;       // Vector de nodos
   size_t maxNodos,    // Tamaño del vector
          numNodos;    // Tamaño del árbol
   nodo libre;         // Lista de celdas libres
```

```
   bool valido(nodo n) const;
}; // class Agen

// Definición del nodo nulo
template <typename T>
const typename Agen<T>::nodo Agen<T>::NODO_NULO{SIZE_MAX};

//
// Método privado
//
template <typename T>
inline bool Agen<T>::valido(nodo n) const
{ // Comprobar si n es un nodo del árbol.
   return !vacio() &&
          n < maxNodos &&                    // Celda del vector
          (n == 0 || nodos[n].padre != NODO_NULO); // ocupada.
}
```

```
template <typename T>
Agen<T>::Agen(size_t maxNodos) :
   nodos{new celda[maxNodos]}, // Listas de hijos vacías.
   maxNodos{maxNodos},
   numNodos{0}
{
   if (maxNodos > 1) { // Crear la lista de celdas libres,
      // para insertar descendientes de la raíz,
      // enlazadas por el hijo izquierdo.
      libre = 1; // Primera libre, pues la 0 se reserva para la raíz.
      for (nodo n = 1; n < maxNodos; ++n) {
         Lista<nodo>& Lh = nodos[n].hijos;
         Lh.insertar(n + 1, Lh.primera()); // Añadir nodo n+1
         #ifndef NDEBUG
         // Sólo para comprobar precondiciones durante depuración.
         nodos[n].padre = NODO_NULO;   // Marcar celda libre.
         #endif
      }
   }
}
```

```
template <typename T>
inline void Agen<T>::insertarRaiz(const T& e)
{
   assert(maxNodos > 0);
   assert(vacio());

   nodos[0] = {e, NODO_NULO}; // Asigna lista de hijos vacía.
   numNodos = 1;
}

template <typename T>
void Agen<T>::insertarHijoIzqdo(nodo n, const T& e)
{
   assert(tama() < tamaMax());
   assert(valido(n));

   // Sacar la primera celda de la lista de libres.
   nodo hizqdo = libre;
   libre = nodos[libre].hijos.elemento(nodos[libre].hijos.primera());
   // Añadir al vector el nuevo hijo de n...
   nodos[hizqdo] = {e, n}; // Asigna lista de hijos vacía.
   //... e insertarlo al inicio de la lista de hijos de n.
   Lista<nodo>& Lh = nodos[n].hijos; // Lista de hijos de n.
   Lh.insertar(hizqdo, Lh.primera());
   ++numNodos;
}
```

```
template <typename T>
void Agen<T>::insertarHermDrcho(nodo n, const T& e)
{
   nodo hedrcho;

   assert(tama() < tamaMax());
   assert(valido(n));
   assert(n != 0);   // n no es la raíz.

   // Sacar la primera celda de la lista de libres.
   nodo hedrcho = libre;
   libre = nodos[libre].hijos.elemento(nodos[libre].hijos.primera());
   // Añadir al vector el nuevo nodo, cuyo padre es el de n,...
   nodos[hedrcho] = {e, nodos[n].padre}; // Lista de hijos vacía.
   //... e insertarlo en la lista de hijos del padre siguiendo a n.
   Lista<nodo>& Lhp = nodos[nodos[n].padre].hijos; // Lista de hijos
                                                   // del padre.
   Lista<nodo>::posicion p = Lhp.primera();
   while (n != Lhp.elemento(p)) p = Lhp.siguiente(p);
   Lhp.insertar(hedrcho, Lhp.siguiente(p));
   ++numNodos;
}
```

```
template <typename T>
void Agen<T>::eliminarHijoIzqdo(nodo n)
{
   assert(valido(n));
   Lista<nodo>& Lh = nodos[n].hijos; // Lista de hijos.
   assert(!Lh.vacia());              // n no es hoja.
   hizqdo = Lh.elemento(Lh.primera());
   assert(nodos[hizqdo].hijos.vacia()); // Hijo izqdo. de n es hoja.

   // Eliminar primer nodo de la lista de hijos de n.
   Lh.eliminar(lh.primera());
   // Añadir hizqdo al inicio de la lista de libres.
   nodos[hizqdo].hijos.insertar(libre, nodos[hizqdo].hijos.primera());
   libre = hizqdo;
   #ifndef NDEBUG
   // Sólo para comprobar precondiciones durante depuración.
   nodos[hizqdo].padre = NODO_NULO; // Marcar celda libre.
   #endif
   --numNodos;
}
```

```
template <typename T>
void Agen<T>::eliminarHermDrcho(nodo n)
{
   assert(valido(n));
   assert(n != 0); // n no es la raíz.
   // Buscar hermano drcho. de n en la lista de hijos del padre.
   Lista<nodo>& Lhp = nodos[nodos[n].padre].hijos;
   Lista<nodo>::posicion p = Lhp.primera();
   while (n != Lhp.elemento(p)) p = Lhp.siguiente(p);
   p = Lhp.siguiente(p);
   assert(p != Lhp.fin());   // n tiene hermano drcho.
   nodo hdrcho = Lhp.elemento(p);
   assert(nodos[hdrcho].hijos.vacia()); // Herm. drcho de n es hoja.

   // Eliminar hermano drcho. de n en la lista de hijos del padre.
   Lhp.eliminar(p);
   // Añadir hdrcho al inicio de la lista de libres.
   nodos[hdrcho].hijos.insertar(libre, nodos[hdrcho].hijos.primera());
   libre = hdrcho;
   #ifndef NDEBUG
   // Sólo para comprobar precondiciones durante depuración.
   nodos[hdrcho].padre = NODO_NULO; // Marcar celda libre.
   #endif
   --numNodos;
}
```

```
template <typename T>
inline void Agen<T>::eliminarRaiz()
{
   assert(numNodos == 1);
   numNodos = 0;
}

template <typename T>
inline bool Agen<T>::vacio() const
{ return numNodos == 0; }

template <typename T>
inline size_t Agen<T>::tama() const
{ return numNodos; }

template <typename T>
inline size_t Agen<T>::tamaMax() const
{ return maxNodos; }
```

```
template <typename T>
inline const T& Agen<T>::elemento(nodo n) const
{
   assert(valido(n));
   return nodos[n].elto;
}

template <typename T>
inline T& Agen<T>::elemento(nodo n)
{
   assert(valido(n));
   return nodos[n].elto;
}

template <typename T>
inline typename Agen<T>::nodo Agen<T>::raiz() const
{ return vacio() ? NODO_NULO : 0; }

template <typename T>
inline typename Agen<T>::nodo Agen<T>::padre(nodo n) const
{
   assert(valido(n));
   return nodos[n].padre;
}
```

```
template <typename T>
inline typename Agen<T>::nodo Agen<T>::hijoIzqdo(nodo n) const
{
   assert(valido(n));
   Lista<nodo>& Lh = nodos[n].hijos;
   return Lh.vacia() ?   // ¿n es hoja?
      NODO_NULO : Lh.elemento(Lh.primera());
}

template <typename T>
typename Agen<T>::nodo Agen<T>::hermDrcho(nodo n) const
{
   assert(valido(n));
   if (n == 0)   // n es la raíz.
      return NODO_NULO;
   else {
      // Buscar hermano drcho. de n en la lista de hijos del padre.
      Lista<nodo>& Lhp = nodos[nodos[n].padre].hijos;
      Lista<nodo>::posicion p = Lhp.primera();
      while (n != Lhp.elemento(p)) p = Lhp.siguiente(p);
      p = Lhp.siguiente(p);
      return p == Lhp.fin() ?   // ¿n tiene hermano drcho.?
         NODO_NULO : Lhp.elemento(p);
   }
}
```

```
template <typename T>
Agen<T>::Agen(const Agen& A) :
   Agen{A.maxNodos} // Garantiza destrucción de copia incompleta de A.
{
   if (!A.vacio()) {
      for (nodo n = 0; n < maxNodos; ++n)
         nodos[n] = A.nodos[n];
      numNodos = A.numNodos;
      libre = A.libre;
   }
}

template <typename T>
inline Agen<T>& Agen<T>::operator =(const Agen& A)
{
   Agen B{A};
   std::swap(nodos, B.nodos);
   std::swap(maxNodos, B.maxNodos);
   std::swap(numNodos, B.numNodos);
   std::swap(libre, B.libre);
   return *this;
}
```

```
template <typename T>
inline Agen<T>::~Agen()
{
   delete[] nodos; // También destruye las listas de hijos.
}

#endif // AGEN_LIS_H
```